Samjhana Ghimire Acharya
HCI- 5840
Fall 2026

Safe Driving Buddy (specification document)

General description of the project

Safe Driving Buddy is a program designed to help drivers learn about accident-prone areas before traveling to unfamiliar locations in the United States. The United States has a strong road-trip culture, and many people travel to unfamiliar places every year. However, drivers are often unaware of accident-prone areas, dangerous intersections, or roads with a high record of traffic crashes. Safe Driving Buddy is a web application designed to help drivers make informed decisions before and during their trips. Users can enter a city and state, and the application will display a map showing traffic accident locations and other relevant safety information.
I will use the NHTSA Crash API to obtain traffic crash data for locations in the United States. For the initial version, Safe Driving Buddy will use a simple command-line interface (CLI). The program retrieves available crash data from the NHTSA FARS API, stores the data in JSON files, and displays crash information in the terminal. This will allow me to focus on the main functionality before developing a web-based interface.

Safe Driving Buddy would use a web-based GUI. I plan to use Flask for the web application and a simple frontend where users can enter a city and state and view crash information on an interactive map.

Task Vignettes (User activity "flow")

Enter a Travel Location:
Users plan to drive to a city they are not familiar with and want to learn about traffic crashes so they can build a mental model for the drive. The user starts Safe Driving Buddy in the command line. The program loads available crash data from previously retrieved JSON files and prepares the data for analysis.

Technical details:

- Input: Crash data stored in JSON files obtained from the NHTSA FARS API.
- Version 1 uses a command-line interface (CLI) in Python.
- The program reads and processes crash data from the JSON files.
- The crash records are organized and prepared for analysis and display.

Retrieve and Process Crash Data:
Safe Driving Buddy retrieves available crash data from the NHTSA FARS API and stores the results in JSON files. The program processes the data and organizes crash records for analysis and display.

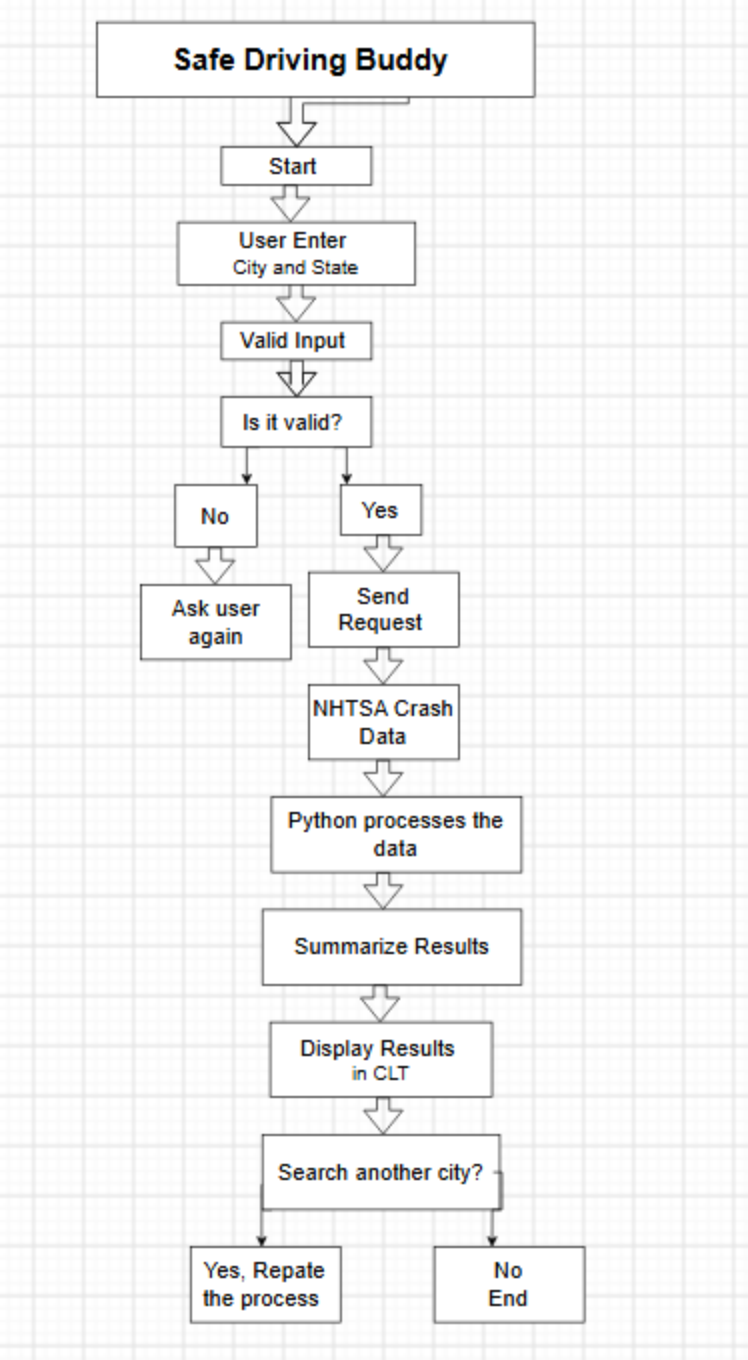
Technical details:

- The program sends a request for crash data using the location information.
- Data is retrieved from the NHTSA Crash API.
- Python processes and organizes the returned data.

View Crash Information
Once the crash information has been processed, Safe Driving Buddy displays the results in the terminal. The user might see information such as the number of crashes found and available crash-location information. The results give the user a better understanding of traffic safety concerns in the area before traveling.
Technical details:

- Output is displayed in the VS Code terminal for Version 1.
- The program summarizes the crash information in an easy-to-read format.
- Basic analysis could identify locations with multiple crashes.
- The user can search another city and state if desired.

Technical "flow"
The Python program retrieves crash data from the NHTSA FARS API, saves it as JSON files, processes it, and displays crash summaries in the terminal. Future versions may include city-based searches, a web interface, and interactive map visualizations.



(Self) asseseemnt

The biggest change was simplifying the first version. Instead of starting with a web application and interactive map, I will begin with a simple CLI and add more features later.

I feel fairly confident that I can implement the project. Since I am still learning Python, breaking the project into smaller steps will make it more manageable.

The biggest challenge was obtaining usable crash data from the NHTSA API. While I successfully retrieved 2024 data, I hit limitations when trying to access data from other years because the data was unavailable and I received 403 Forbidden errors. Moving forward, the challenge will be improving data processing and implementing additional user-friendly features.

I may need help with accessing and processing the crash data in Python. Since working with an external API is new to me, I may also need guidance on choosing the appropriate data and keeping the first version of the project manageable.